

QA LAB BRASIL

Guia de referência - QA Lab Brasil

Engenharia de Software

Como software é concebido, desenvolvido, integrado, implantado, operado e evoluído.

Como um software nasce

Um produto de software normalmente evolui de uma necessidade para requisitos, desenho da solução, desenvolvimento, testes, entrega, operação e manutenção. QA pode contribuir desde a análise do requisito até a observação do comportamento em produção.

SDLC e STLC

SDLC significa Software Development Life Cycle. STLC significa Software Testing Life Cycle. O primeiro descreve o ciclo de desenvolvimento; o segundo organiza atividades de teste como análise, planejamento, desenho, preparação, execução e encerramento.

Requisitos funcionais e não funcionais

Requisitos funcionais descrevem o que o sistema deve fazer. Não funcionais tratam atributos como desempenho, segurança, acessibilidade, confiabilidade e usabilidade. QA deve questionar ambiguidades e transformar requisitos em condições verificáveis.

Arquiteturas que o QA encontra

Monólitos, arquitetura em camadas, cliente-servidor, microsserviços, sistemas orientados a eventos e serverless alteram onde falhas podem surgir. Entender componentes e dependências melhora a investigação.

Frontend, backend e integrações

Frontend é a camada de interação com o usuário; backend executa regras e processamento; APIs, filas e eventos conectam serviços. Um defeito visível na tela pode ter origem em qualquer uma dessas camadas.

Qualidade arquitetural

Performance, segurança, confiabilidade, manutenibilidade, compatibilidade e escalabilidade devem ser consideradas desde o desenho. Testabilidade e observabilidade também influenciam a capacidade de detectar e diagnosticar problemas.

Dívida técnica, refactoring e manutenção

Dívida técnica representa decisões que aceleram uma entrega, mas podem aumentar custo futuro. Refactoring melhora a estrutura interna sem intenção de alterar comportamento; mesmo assim pode exigir regressão baseada em risco.

Visão avançada para QA

Um QA experiente não precisa ser arquiteto, mas deve conseguir ler fluxos, identificar pontos de falha, discutir riscos, dependências, contratos, dados e mecanismos de recuperação.

QA Tip: Ao investigar um problema, desenhe o fluxo dos dados antes de assumir que a causa está na interface.